
boolean



JS - Istruzioni Condizionali

Sarò io a dettare le condizioni

Istruzioni Condizionali

E se?

Se vogliamo far “accadere una cosa” solo in un caso specifico?

Le **istruzioni condizionali** eseguono un certo blocco di codice **se si verifica una ben precisa condizione**.

```
1 if (condizione) {
2
3   // blocco di istruzioni
4
5 }
```

Se si verifica la (condizione) {

// allora viene eseguito questo blocco di istruzioni (notare le graffe)

}

Istruzioni Condizionali

E se?

Se vogliamo far accadere una cosa solo in un caso specifico "altrimenti un'altra?"

Ci sono situazioni in cui vogliamo eseguire un certo blocco di codice **oppure un altro.**

```
1 if (condizione) {
2
3  // blocco di istruzioni 1
4
5 } else {
6
7  // blocco di istruzioni 2
8
9 }
```

Se si verifica la (condizione) {

// allora viene eseguito questo blocco di istruzioni 1 (notare le graffe)

} **altrimenti** {

// viene eseguito questo blocco di istruzioni 2 (notare le graffe)

}

Istruzioni Condizionali

E se... Aggiungiamo condizioni?

Possiamo aggiungere altre condizioni **esclusive** con **else if**.

La parte “else” non è obbligatoria.

```
1  if (condizione) {
2
3   // blocco di istruzioni 1
4
5  } else if (altra condizione ) {
6
7   // blocco di istruzioni 2
8
9  } else {
10
11  // blocco di istruzioni 3
12
13 }
14
```

Se si verifica la (condizione) {

// allora viene eseguito questo blocco di istruzioni 1

} **altrimenti se** (altra condizione) {

// viene eseguito questo blocco di istruzioni 2

} **altrimenti** { // se entrambe le precedenti sono false

// viene eseguito questo blocco di istruzioni 3

}

Le condizioni

Cosa è una condizione

Una **condizione** è la verifica che un valore corrisponda a:

true o **false**
(vero = condizione verificata)
o
(falso = condizione non verificata).

```
1  if (condizione) {
2
3   // blocco di istruzioni 1
4
5  } else if (altra condizione ) {
6
7   // blocco di istruzioni 2
8
9  } else {
10
11  // blocco di istruzioni 3
12
13 }
14
```

Operatori relazionali

Come confrontiamo due valori

Come otteniamo questo valore Booleano?
Confrontando due valori con gli **operatori relazionali**

== uguaglianza	Se x = 3, allora	x == 4 è falso
!= diversità	Se x = 3, allora	x != 4 è vero
> maggiore di	Se x = 3, allora	x > 4 è falso
< minore di	Se x = 3, allora	x < 4 è vero
>= maggiore o uguale	Se x = 3, allora	x >= 4 è falso
<= minore o uguale	Se x = 3, allora	x <= 4 è vero

Operatori relazionali

Come confrontiamo due valori

Trova la differenza!



== uguaglianza	Se x = 3, allora	x == 4 è falso x == "3" è vero
!= diversità	Se x = 3, allora	x != 4 è vero x != "3" è falso
=== identità uguaglianza di valore e tipo	Se x = 3, allora	x === 3 è vero x === "3" è falso
!== non identità disuguaglianza di valore e tipo	Se x = 3, allora	x !== 3 è falso x !== "3" è vero

Operatori Logici

Possiamo unire più condizioni?

Gli operatori logici consentono la **combinazione di espressioni booleane**.

JavaScript prevede

- **due operatori binari** (che richiedono due operandi)
- **un operatore unario** (che richiede un operando)

Operatori Logici

Possiamo unire più condizioni?

Operatori Binari

&& and, “e logico” restituisce vero se e soltanto se entrambi gli operandi sono veri	Se x = 3, y = 4 allora	(x < 4 && y > 5) è falso
|| or, “o logico” restituisce vero se almeno uno degli operandi è vero	Se x = 3, y = 4 allora	(x < 4 || y > 5) è vero

Operatori Logici

Possiamo unire più condizioni?

Una condizione può essere formata da uno o più confronti con la combinazione degli:

operatori relazionali e **operatori logici**

```
1 if (x === 3) {
2
3  // Questo blocco viene eseguito
4  // solo se x è uguale a 3
5
6  alert("X è uguale a 3");
7
8 }
```

```
1 if ((x === 3) && (y === 4)) {
2
3  // Questo blocco viene eseguito
4  // solo se x è uguale a 3 e Y uguale a 4
5
6  alert("X === 3 e Y === 4");
7
8 }
```

Operatori Logici

Possiamo unire più condizioni?

Operatore Unario

! not, negazione cambia il valore di verità dell'operando	Se x = 3, y = 4 allora	!(x > y) è vero

🤔 Ma quando mai vorrò sapere il contrario di qualcosa ? ...

Operatori Logici - NOT

Ma quando mai vorrò sapere il contrario di qualcosa?

La negazione ci da uno strumento per rendere **più espressivo** il nostro codice, e quindi **non essere vincolati ad un singolo modo** di porre un concetto.
Come in qualsiasi **lingua** parlata, è necessario anche in **JS** in quanto **linguaggio** di programmazione.

In questo esempio, per controllare se un utente non è loggato, siamo vincolati all'uso di un *else.*

```
let utenteLoggato = false;

if (utenteLoggato) {

 // questo blocco di istruzioni non sarà eseguito
 // perchè false === false

} else {

 // sarà eseguito questo blocco di istruzioni

}

```

Operatori Logici - NOT

Ma quando mai vorrò sapere il contrario di qualcosa?

In questo altro esempio il **NOT** ci libera dalla costrizione precedente, permettendoci di esprimere la stessa cosa ma diversamente.

Possiamo quindi partire dalla condizione che riteniamo più adatta e organizzare il codice come meglio crediamo.

```
let utenteLoggato = false;

if (!utenteLoggato) {

 // questo blocco di istruzioni sarà eseguito
 // perchè !false === true
}

// ... 100k LOC dopo...

if (utenteLoggato) {

 // questo blocco di istruzioni non sarà eseguito
 // perchè false === false
}
```



Non c'è un modo giusto ed uno sbagliato in assoluto,
ma modi diversi di poter **affrontare e risolvere** i problemi.



LIVE CODING

Random in JS

Può essere utile avere un numero random, cioè generato dal computer

```
1 // Restituisce un valore decimale da 0 a 1
  (non compreso)
2
3 Math.random();
4
```

```
1 // Restituisce un numero tra 0 e 9
2 Math.floor(Math.random() * 10);
3
4 // Restituisce un numero tra 1 e 100
5 Math.floor(Math.random() * 100) + 1;
```

Math.floor è **l'arrotondamento per difetto**, cioè arrotonda sempre al numero intero più piccolo.
Di contro, Math.ceil è l'**arrotondamento per eccesso** e arrotonda sempre al numero intero più grande.

https://www.w3schools.com/js/js_random.asp

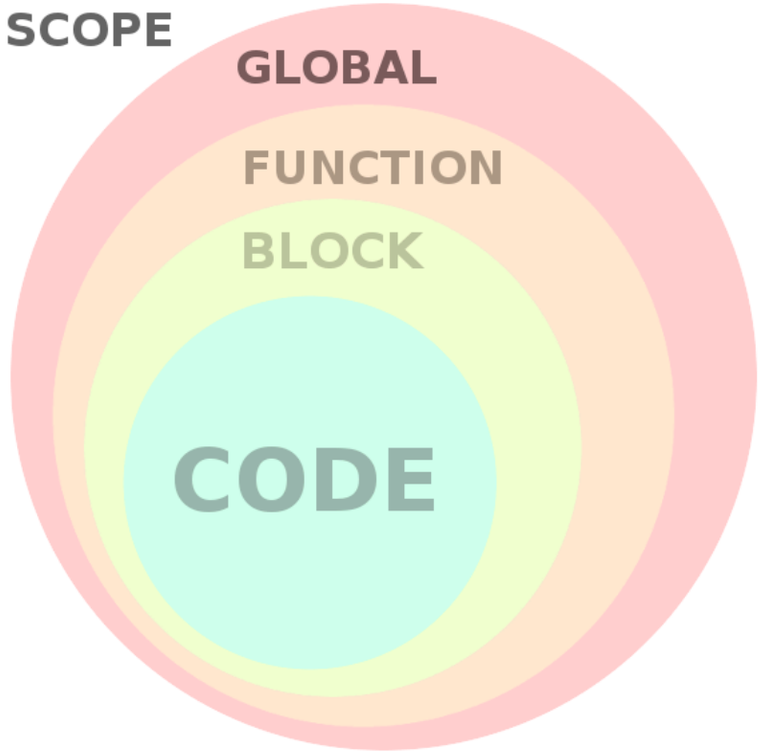


LIVE CODING

Variable Scope

Anche detto ambito di visibilità (di una variabile)

Lo scope non è altro che il **contesto all'interno del quale si può richiamare una certa variabile.**



Per ora ci basterà sapere che una variabile **può essere definita a livello globale, in una funzione, o in un blocco di codice (ES6+).**

Variable Scope

Ma quindi come si comporta una variabile dichiarata in un blocco if?

Vediamo insieme qualche esempio

Una variabile definita esternamente ad una funzione o ad un qualsiasi blocco di codice **ha uno scope globale** ed è accessibile da qualsiasi blocco.

Una variabile definita internamente ad un qualsiasi blocco di codice **ha uno scope di tipo block** ed è accessibile **esclusivamente dall'interno di quel blocco**.

```
1  let validation = true;
2  const wordA = "Word A";
3  let wordB = "Word B";
4
5  if ( validation === true) {
6    let wordC = "Word C";
7    let wordB = "Word Custom B";
8    console.log(wordA);
9    console.log(wordB);
10   console.log(wordC);
11 }
12
13 console.log(wordA);
14 console.log(wordB);
15 console.log(wordC);
```

Variable Scope

Quindi cosa ci restituiranno le queste funzioni console.log()?

```
1  let validation = true;
2  const wordA = "Word A";
3  let wordB = "Word B";
4
5  if ( validation === true) {
6    let wordC = "Word C";
7    let wordB = "Word Custom B";
8    console.log(wordA);
9    console.log(wordB);
10   console.log(wordC);
11 }
12
13 console.log(wordA);
14 console.log(wordB);
15 console.log(wordC);
```



LIVE CODING

Variable Scope

Cerchiamo di capire insieme il motivo di questi risultati

```
let validation = true;
const wordA = "Word A";
let wordB = "Word B";

if ( validation === true) {
  let wordC = "Word C";
  let wordB = "Word Custom B";
  console.log(wordA); // Word A
  console.log(wordB); // Word Custom B
  console.log(wordC); // Word C
}

console.log(wordA); // Word A
console.log(wordB); // Word B
console.log(wordC); // ReferenceError: wordC is not defined
```



ESERCITAZIONE